CA3 – AGENTIC AI

Course / Subject: Agentic AI

Course Title: Autonomous AI System and Agent-based Computing

Course Duration (minimum 45 hours): [To be completed by student]

Faculty: Nilima Zade

Division: C-2

Student Name: Ojas Verma

PRN: 23070122153

Project: SpendWise AI

Final technical report and local demonstration record · 17 October 2026

This report documents the existing local expense tracker after its MongoDB migration. It explains the implemented workflow, source code, tests and evidence. The local application, database and project repository URL were verified. Course certificates and any required public application deployment remain submission items where stated.

1. Introduction to Agentic AI

Objective

The objective is to reduce the effort of recording daily expenses while making each save trustworthy. A user writes a short message, the local language model proposes structured fields, deterministic code validates them, and n8n stores and reads back the record in MongoDB. The dashboard then uses those records for expense history and spending summaries.

The project demonstrates a bounded AI workflow: language understanding is delegated to a model, while business rules, tool access, storage and the final success decision remain explicit in code. It is a local single-user academic prototype.

Key Concepts Understood

Key concepts include structured model output, tool orchestration, validation, persistent state, idempotent requests and feedback. A JSON Schema constrains the shape of the model response; source-based checks prevent fabricated amounts and ambiguous dates. MongoDB stores the durable expense, while a submission journal tracks retry state. The receipt is generated from a matching database readback rather than from the model's text.

2. Understanding of Agentic AI

What is an AI Agent?

An AI agent receives observations, interprets a goal, chooses or follows actions through tools, maintains relevant state, and uses feedback to decide what happens next. In SpendWise, the observation is an expense message, the local model extracts candidate fields, n8n routes the workflow, and MongoDB supplies storage feedback.

The current implementation uses predefined n8n routing rather than an unconstrained agent planning loop. Its autonomy is limited to interpreting the description and selecting a permitted category. Code controls the amount, date, allowed currency and database operations. It does not transfer money, access bank accounts or provide open-ended financial advice.

This separation matters because syntactically valid JSON can still contain incorrect financial data. A schema addresses format; deterministic validation and storage verification address the conditions under which the application can accept and confirm an expense.

3. Courses Completed (multiple courses as applicable)

Course Details

The student will add actual course progress, titles, providers, hours and certificate evidence. No course completion or minimum-hour requirement is claimed in this report. Both original course entries are retained below; add further entries if required.

1. Course Title: [To be completed by student]

Platform / Provider: [To be completed by student]

Course Duration (minimum 45 hours): [To be completed by student]

Completion Evidence: [To be completed by student]

2. Course Title: [To be completed by student]

Platform / Provider: [To be completed by student]

Course Duration (minimum 45 hours): [To be completed by student]

Completion Evidence: [To be completed by student]

4. Problem Statement

Problem Statement

Problem Statement: Everyday expense logging becomes inconvenient when each transaction requires several manual fields. Free-text notes are easier to enter but do not directly support consistent totals, categories and dates. An AI response alone also cannot prove that a transaction was stored.

The project therefore needs to convert one natural-language expense into validated data, preserve the original message, save it reliably and retrieve the same records for history and charts. Missing amounts, ambiguous dates, unsupported currency and service failures must produce an honest clarification or error.

The earlier design used Google Sheets. The final implementation uses local MongoDB in Docker, avoiding the Google OAuth dependency while preserving the existing Ollama model, n8n account, application code and Docker volumes. Historical Sheets data was not imported because access to that history was not verified.

5. Proposed Solution

Project Title

SpendWise AI

Project Description

SpendWise AI is a local personal-finance dashboard with natural-language expense entry, searchable transaction history, category and daily spending charts, and monthly category budgets. React communicates with an Express backend; an authenticated n8n workflow calls Ollama llama3.2:3b, validates the proposed fields, and persists the expense in MongoDB. The frontend displays a save receipt only after the workflow confirms an actual matching stored document.

The final scope supports one positive INR expense per message. It preserves amount, currency, category, description, date and original message, together with an identifier and timestamp. Budgets remain local backend configuration. The product is intended for local demonstration and is not a hosted multi-user finance service.

6. Implementation

Technologies Used

Programming Language: TypeScript and JavaScript; HTML and CSS.

LLM / AI Model: Existing Ollama llama3.2:3b; native JSON Schema response; temperature 0.

Agent Framework: n8n 2.35.6 as the workflow orchestrator with explicit routes and checks.

Tools / APIs: Authenticated n8n webhook, Ollama HTTP API and native n8n MongoDB nodes.

Database / Storage: MongoDB 8.0 in Docker; persistent named volume; local JSON budget and retry files.

Frontend: React 19, Vite 7, TypeScript and lucide-react icons.

Backend: Node.js and Express 5; same-origin local API, validation and retry reconciliation.

Implementation Steps

Architecture and responsibility

Figure 1 separates the browser, backend, automation, model and database. n8n orchestrates the model call before performing storage. All main services run on the local computer; the frontend does not contain the database password or webhook secret.

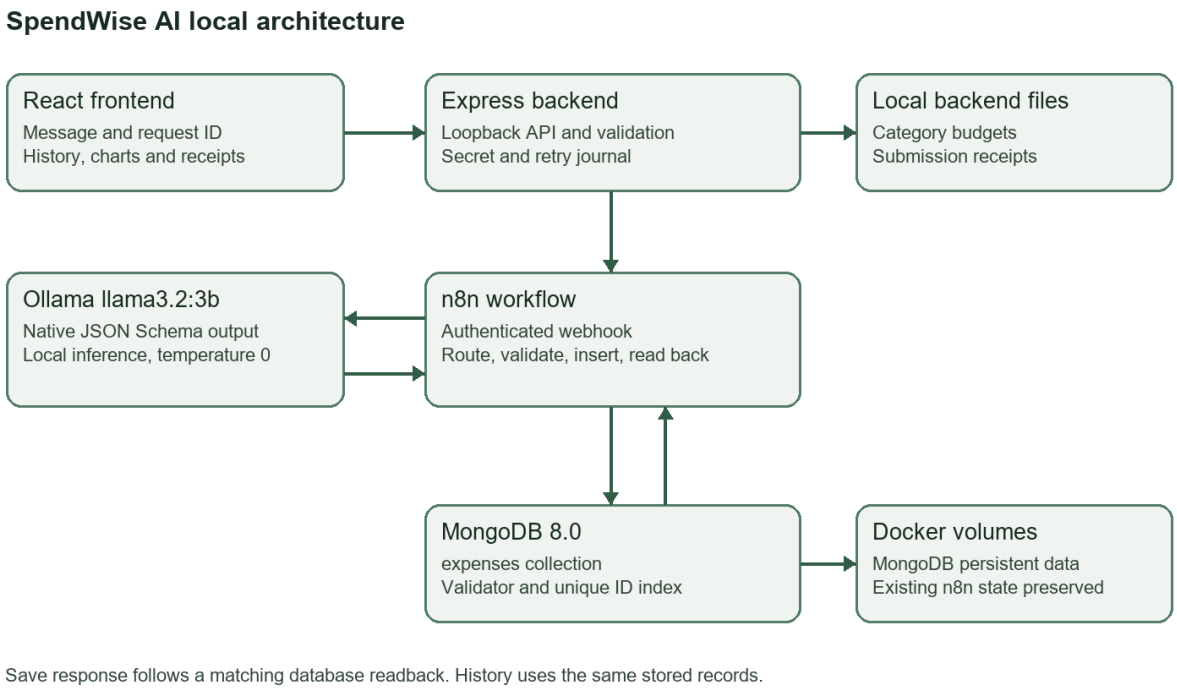


Figure 1. Source-derived architecture diagram. This is an explanatory diagram, not a screenshot.

Workflow implementation

1. Accept the message and a generated request ID through POST /api/expenses. The backend validates request shape and origin, allows one active submission, and writes a pending entry to its local retry journal.

2. Send the request to the authenticated production n8n webhook. The secret is added by the backend from environment configuration. n8n computes the current date in Asia/Kolkata.

3. Call local Ollama /api/chat with the extraction schema, stream disabled, temperature 0 and bounded generation. Parse the returned JSON; invalid JSON becomes a clarification rather than a proposed save.

4. Validate that exactly one positive INR amount occurs in the original message. Reject uncertain or negated spending, missing or multiple amounts, unsupported currency, invalid descriptions/categories and invalid, conflicting or future dates.

5. Resolve dates in code. Supported forms include today, yesterday, day before yesterday, ISO date and day plus English month. An omitted date means today in India; an omitted year uses the current local year. Unsupported date expressions require clarification.

6. Check MongoDB for the request ID. An existing matching record may be returned without another insertion. Otherwise insert the canonical fields and read the record back. A genuine MongoDB _id and matching ID, date, amount, currency, category, description and original message are required before status saved.

7. Validate the receipt in the backend, update the journal and return it to React. React refreshes real history after a confirmed save. GET /api/expenses follows the workflow's read route to MongoDB; dashboard totals and chart groups derive from those retrieved records.

Database contract

The primary collection is spendwise.expenses. Documents use the canonical names below. The established UI contract retains cost, descr and msg aliases so working frontend components can be preserved. normalizeRows performs that mapping.

Stored field	Meaning and constraints
id	Submission identifier; unique indexed string.
amount	Positive INR number, at most two decimal places in application validation; maximum 10,000,000.
currency	INR only.
category	One of eight allowed expense categories.
description	Nonempty short description, maximum 160 characters.
date	YYYY-MM-DD expense date; calendar and future-date checks in code.
originalMessage	Original submitted text, maximum 2,000 characters.
createdAt	Creation timestamp as an ISO string.
_id	MongoDB ObjectId used as evidence of a genuine document.

MongoDB has a strict required-field/type validator, a unique id index and a date/category index. The validator rejects invalid document types and ranges, while application rules check source evidence, decimal precision and calendar semantics. Budgets are kept in .local/budgets.json; the MongoDB retry journal is .local/requests-mongodb.json.

Structured output and meaningful source excerpts

The following listings are short exact extracts from the final source. They demonstrate format constraints, validation, integration, persistence, APIs and frontend confirmation. The complete source files remain authoritative; evidence/code contains the same excerpts and their source hashes.

Listing 1. Expense extraction schema

Source: shared/domain.ts, lines 6–12. Exact excerpt; surrounding code is omitted.

```
export const extractionSchema = {
  type: 'object', additionalProperties: false,
  properties: {
    cost: { type: ['number', 'null'] }, date: { type: ['string', 'null'] },
    currency: { type: ['string', 'null'] }, category: { type: 'string', enum: [...categories] },
    descr: { type: 'string' }
  }, required: ['cost', 'date', 'currency', 'category', 'descr']
```

The schema defines required fields and allowed categories. Listing 2 passes this schema as Ollama’s format field with temperature 0; the model proposes data but never confirms storage.

Listing 2. Native structured-output request

Source: shared/domain.ts, lines 53–54. Exact excerpt; surrounding code is omitted.

```
export function extractionRequest(message: string, today: string, model: string) {
  return { model, stream: false, format: extractionSchema, options: { temperature: 0, num_predict:
240, num_ctx: 2048 }, messages: [
```

The request sends the actual schema as format and disables streaming. Temperature 0 and generation/context limits make extraction bounded; subsequent JSON decoding and business validation remain necessary.

Listing 3. Date and description validation

Source: shared/domain.ts, lines 68–71. Exact excerpt; surrounding code is omitted.

```
  const date = resolveDate(message, today);
  if (!date || date > today) return { message: 'Please specify the expense date as YYYY-MM-DD or
“yesterday”. Future expenses cannot be recorded.' };
  if (typeof p.descr !== 'string' || !p.descr.trim() || p.descr.length > 160 ||
!categories.includes(p.category as Category)) return { message: 'What was this expense for? Please
add a short description.' };
  return { expense: { id, date, cost: p.cost, currency: 'INR', category: p.category as Category,
descr: p.descr.trim(), msg: message } };
```

The date is resolved from the original message, checked against today, and combined with a permitted category and bounded description. Earlier checks in the same function require one supported positive amount from the message and reject uncertain input.

Listing 4. Authenticated workflow integration

Source: server/integration.ts, lines 7–11. Exact excerpt; surrounding code is omitted.

```
  try {
    response = await fetch(`${config.n8n}/webhook/${webhookPath}`, {
      method: 'POST', headers: { 'Content-Type': 'application/json', 'X-SpendWise-Key':
config.secret },
      body: JSON.stringify(body), signal: AbortSignal.timeout(readOnly ? 15000 : 150000)
    });
```

The backend adds the private webhook header and applies a bounded timeout. The browser calls the backend and never receives this secret. Subsequent code handles HTTP and malformed-response failures conservatively.

Listing 5. Readback before a confirmed save

Source: scripts/build-workflow.mjs, lines 46–51. Exact excerpt; surrounding code is omitted.

```
code('Save confirmed',`if($input.all().some(i=>i.json.error)) throw new Error('MongoDB save could
not be read back');
const expected=$('Validate expense').first().json.expense;
const stored=$input.all().find(i=>i.json._id&&i.json.id===expected.id);
const actual=stored&&SpendWise.normalizeRows([stored.json]).expenses[0];
if(!actual||['id','date','cost','currency','category','descr','msg'].some(k=>actual[k]!==expected[k]
)) throw new Error('MongoDB save confirmation mismatch');
return [{json:{status:'saved',expense:actual}}];`,[2620,460]);
```

This workflow-generator excerpt builds the Save confirmed Code node. A real MongoDB identity is required and each returned business field must match the validated expense. Every MongoDB node stops the workflow on database errors.

Listing 6. Strict collection and indexes

Source: docker/mongo-init.js, lines 21–25. Exact excerpt; surrounding code is omitted.

```
for (const name of ['expenses','expense_integration_tests']) {
  appDb.createCollection(name, { validator: { $jsonSchema: schema }, validationLevel: 'strict',
validationAction: 'error' });
  appDb.getCollection(name).createIndex({ id: 1 }, { unique: true, name: 'unique_request_id' });
  appDb.getCollection(name).createIndex({ date: -1, category: 1 }, { name: 'date_category' });
}
```

Initialization creates the personal and isolated test collections with validation and indexes. It runs only on a new empty volume; existing initialized data is reused.

Listing 7. History API backed by the database

Source: server/app.ts, lines 33–35. Exact excerpt; surrounding code is omitted.

```
  app.get('/api/expenses', async (_req, res, next) => {
    try { res.json({ ...await records(), source: 'mongodb', today: localDate(new Date(),
config.timezone) }); } catch (e) { next(e); }
  });
```

records() invokes the workflow read route and normalizes its actual documents. The API declares source mongodb and returns India's current date; read failures become errors rather than empty mock data.

Listing 8. Frontend receipt and refresh

Source: src/components.tsx, lines 25–30. Exact excerpt; surrounding code is omitted.

```
      const result = await api.add(text, request.current.id);
      if (result.status === 'saved') {
        setItems(old => [...old, { role: 'assistant', kind: 'saved', text: 'Saved to MongoDB',
expense: result.expense }]);
        setMessage(''); request.current = null; onSaved();
      } else { setItems(old => [...old, { role: 'assistant', kind: result.status, text:
result.message }]); if (result.status === 'clarification') request.current = null; }
    } catch (error) { setItems(old => [...old, { role: 'assistant', kind: 'error', text: (error as
Error).message }]); }
```

The UI announces Saved to MongoDB only for a saved result. A clarification or error uses its own message; the dashboard refresh callback runs only after the confirmed receipt.

Reliability and credential protection

A unique ID, request fingerprint and local journal protect retries. A repeated ID with different text is rejected. A previous attempt must be reconciled against live MongoDB before returning a cached receipt; an unknown write is not blindly resubmitted. The unique index provides a second duplicate barrier. This is local single-process protection, not a distributed exactly-once guarantee.

The private .env holds the webhook secret and separate MongoDB root/application credentials. .env.example contains configuration keys without passwords. n8n uses a scoped readWrite user for the SpendWise database; its editor login is a separate credential. The MongoDB host port binds to 127.0.0.1:27018, and n8n uses mongodb:27017 on the Docker network. The app binds locally and rejects cross-site writes.

SpendWise retains spendwise_n8n_data on port 5679 and uses the new spendwise_mongodb_data volume. The original n8n service on port 5678, existing Google credentials and spreadsheet were preserved. Private backup directories can contain secrets and are excluded from the submission evidence. Do not delete volumes or regenerate credentials to troubleshoot.

Testing and observed results

Existing verification records dated 17 October 2026 were reused for this documentation step. No application rebuild, model download or repeat database-outage test was needed. The recorded build passed TypeScript and Vite production compilation. Fifteen regression tests passed; they use temporary files and simulated upstream responses where appropriate. The integration checks below used actual Ollama, n8n and MongoDB.

Check	Observed result	Evidence
Build and regression tests	Build passed; 15 tests passed.	docs/VERIFICATION.md
Full service flow	Four real test records; INR 1,070 total.	mongodb-e2e-results.json
Stored field checks	Independent driver matched required fields.	test-mongodb.ts and result log
Retry and ID conflict	Replay retained one record; conflicting text rejected.	E2E and regression tests
Clarification cases	Missing amount, dollars and conflicting dates clarified.	E2E result log
Database constraints	Duplicate ID: error 11000; negative amount: error 121.	Recorded E2E test assertions
Database outage	No save success; history HTTP 503.	Recorded E2E result log
Restart persistence	All four test documents survived restart.	E2E result log
Browser test	Coffee receipt observed; 5 records, INR 1,110, average INR 222.	mongodb-browser-results.json
Search and budget	Coffee filtered to one; Transport 320 of 500, 64%.	Browser verification record
Current local snapshot	1 stored lunch record; INR 250 total/average; services healthy.	Current query and Figures 2–7

The earlier four/five-record tests used expense_integration_tests, a visibly labeled isolated workspace and temporary budget files. Those test IDs were cleaned up. The final screenshots show the currently existing personal collection containing one lunch record, independently read through the MongoDB driver. These are different snapshots, not contradictory totals.

An outage test previously exposed n8n passing original input when MongoDB nodes continued after connection errors. Database nodes now stop on error, and readback requires _id plus matching fields. The complete test passed after this correction. The unavailable historical Information Extractor output was not diagnosed; the replacement native structured-output route was independently verified.

Date filters, sorting, CSV export and responsive CSS are implemented. The recorded browser tests covered save, history, totals, search/category filter and budget progress; they did not exercise every control or physical device. The captured narrow layout is genuine browser evidence, not comprehensive mobile-device certification.

7. Final Project Demonstration

Prototype / Application Link

Verified local application: http://localhost:3000. This is a running local installation, not a public deployment.

Open the local SpendWise application

Open the local published n8n workflow

Public deployment: [Not verified / not supplied]. No public live URL or deployment screenshot is claimed. Local services must be running on this computer for the localhost links to work.

GitHub Repository

Project repository: https://github.com/pratiklakra38/Spendwise

Project source, configuration and submission documents are maintained in the repository provided by the student. It is hosted under pratiklakra38; the report retains Ojas Verma as the supplied student name. Repository hosting is separate from application deployment.

Project source and documentation on GitHub

Project documentation

README and setup instructions

Backend API contract

Detailed verification record

Evidence folder and submission checklist

Documentation links are relative to submission/; keep the package together. The essential architecture, workflow, testing and startup instructions are included in this report.

Demonstration

Figures 2–7 are genuine local captures from 17 October 2026. No personal expense was added for documentation. They show one existing INR 250 lunch record, not proof of a bank payment. Original PNGs and query evidence are retained under evidence/.

Demonstration procedure

Start Docker Desktop and the existing Ollama instance. In the project folder run ./start.ps1 only if the app is stopped. Open the app, check Settings, submit one supported expense, wait for a receipt and inspect its history/details and MongoDB ID. Use the isolated workflow for synthetic academic tests.

Setup and reproducibility

The current computer is configured. Production startup uses docker compose up -d --wait, npm.cmd run build and npm.cmd start. A new checkout needs npm.cmd ci, private .env settings, npm.cmd run setup:mongodb and ./scripts/install-workflow.ps1 as detailed in README. The preserved local n8n image and external volume must already be configured; these commands do not reconstruct private resources.

npm.cmd test runs regression tests. npm.cmd run test:e2e uses the isolated real collection, temporarily stops only SpendWise MongoDB, restores it and cleans its own records. Its --serve mode provides a labeled browser workspace on port 3001. These are reproduction instructions, not a new test-run claim.

Application dashboard evidence

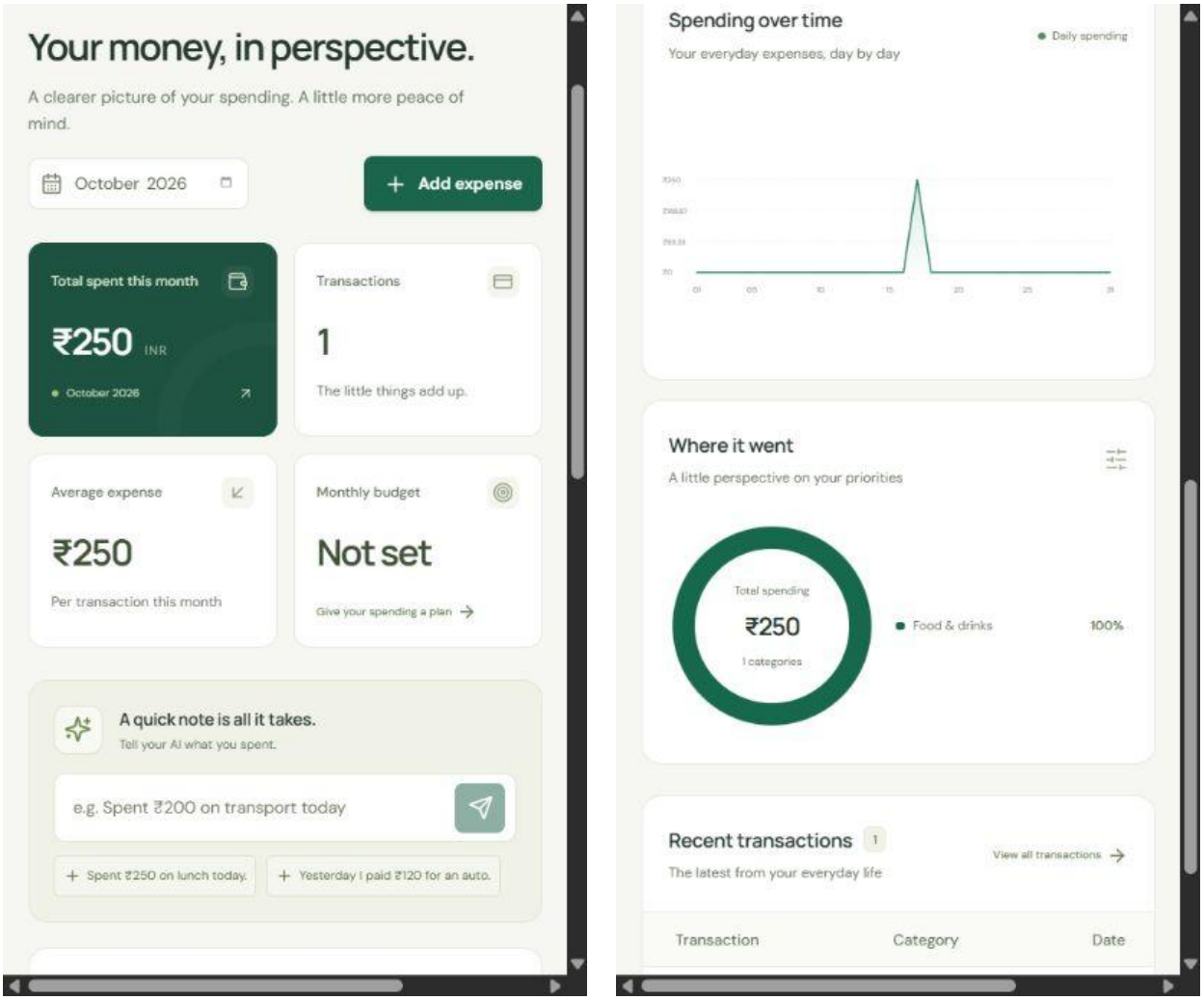


Figure 2. Actual local dashboard summary (left) and daily/category charts (right). October 2026 contains one INR 250 lunch transaction. Narrow browser captures retain the actual responsive layout.

The summary and chart share the same retrieved transaction set. At capture, the transaction count is 1 and total/average are INR 250. The category chart attributes the full amount to Food & drinks.

Expense entry and stored details

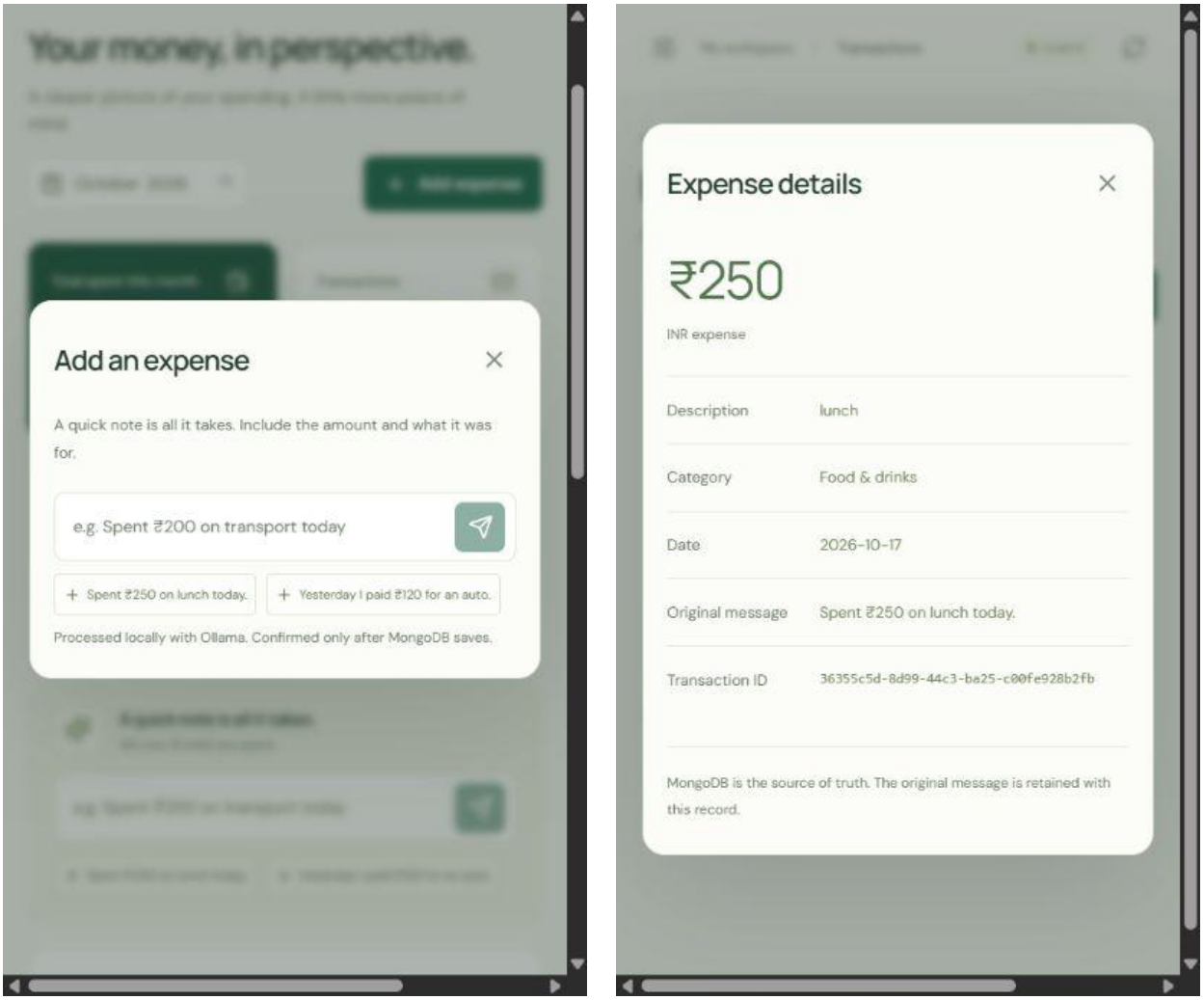


Figure 3. Actual expense-entry dialog (left) and the existing MongoDB-backed lunch detail view (right). The entry dialog is an interface capture, not a new submission or a success receipt.

The detail view retains the original message and transaction ID. The earlier isolated browser test observed a genuine save receipt, as recorded in mongodb-browser-results.json; no reconstructed receipt image is presented here.

Transaction history

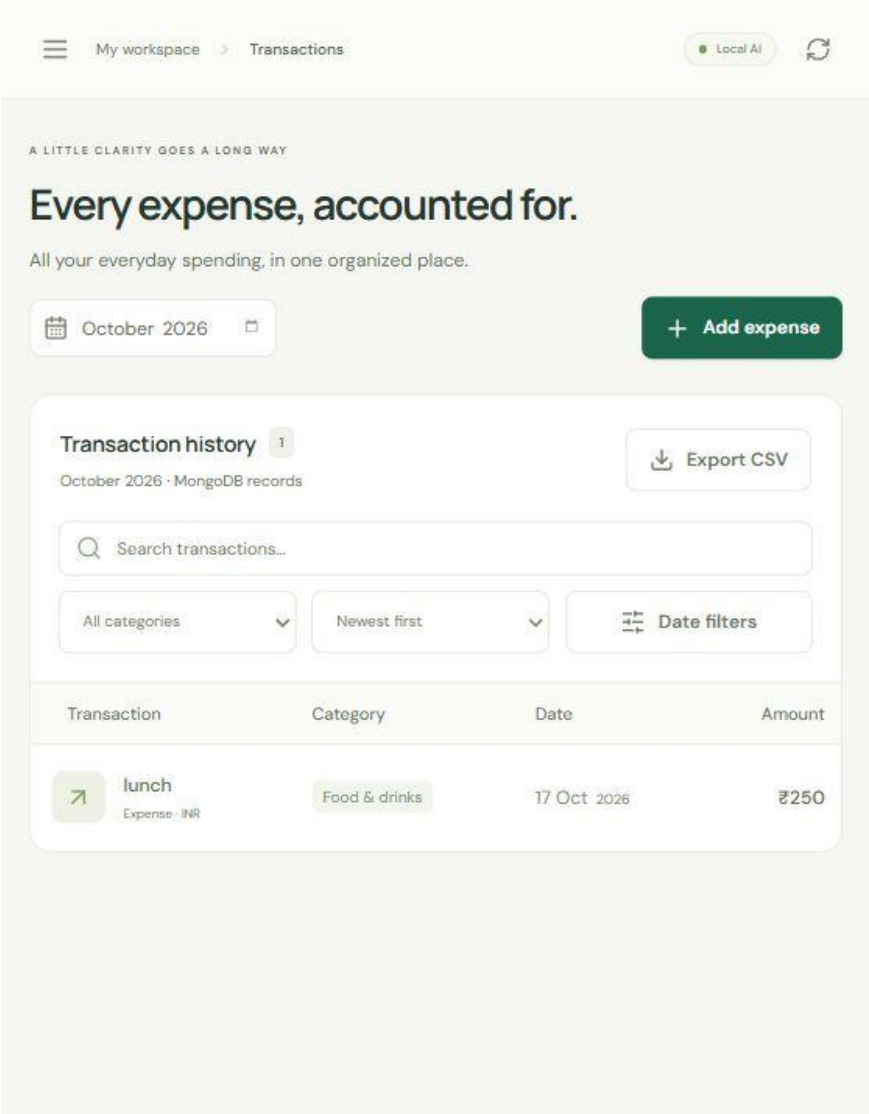


Figure 4. Actual transaction history for October 2026, retrieved from MongoDB. One lunch record is visible; the responsive table can scroll horizontally to reveal additional columns.

The API snapshot reports source mongodb, invalidRows 0 and one transaction. Search/category/date filters and sorting operate on retrieved records. The independent database query in Figure 7 confirms the canonical stored fields.

Published automation workflow

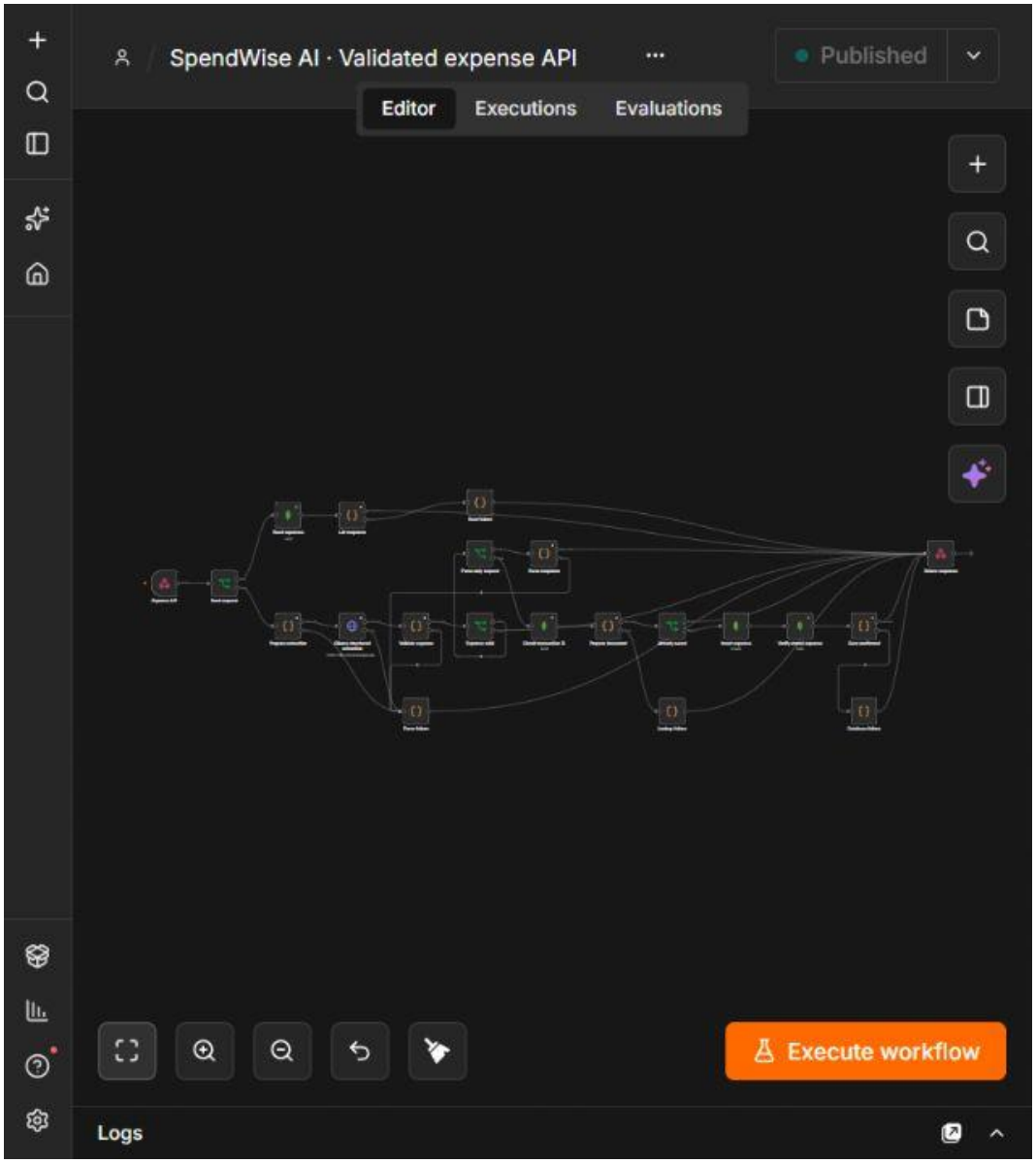


Figure 5. Actual published SpendWiseLocalV1 workflow in the local n8n editor. The full-resolution PNG is supplied separately. Figure 1 and Listings 3–5 explain the routes and database checks at readable scale.

Live local connection status

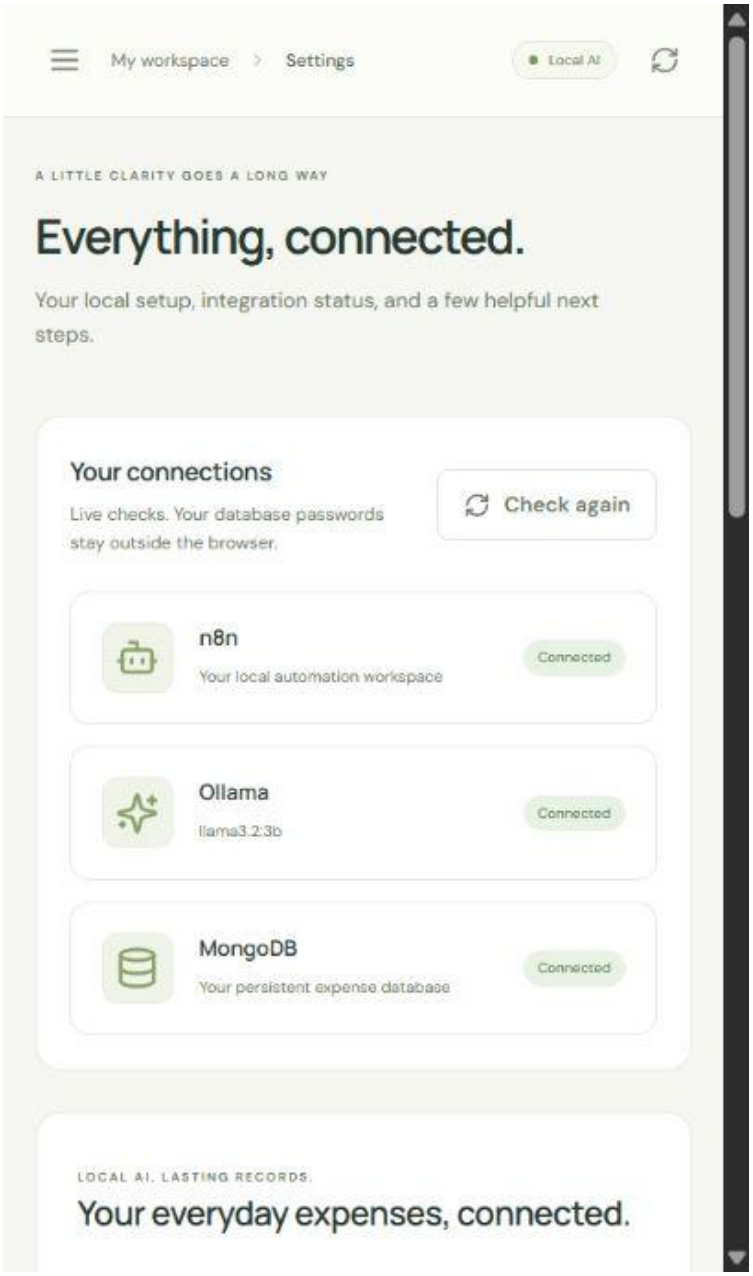
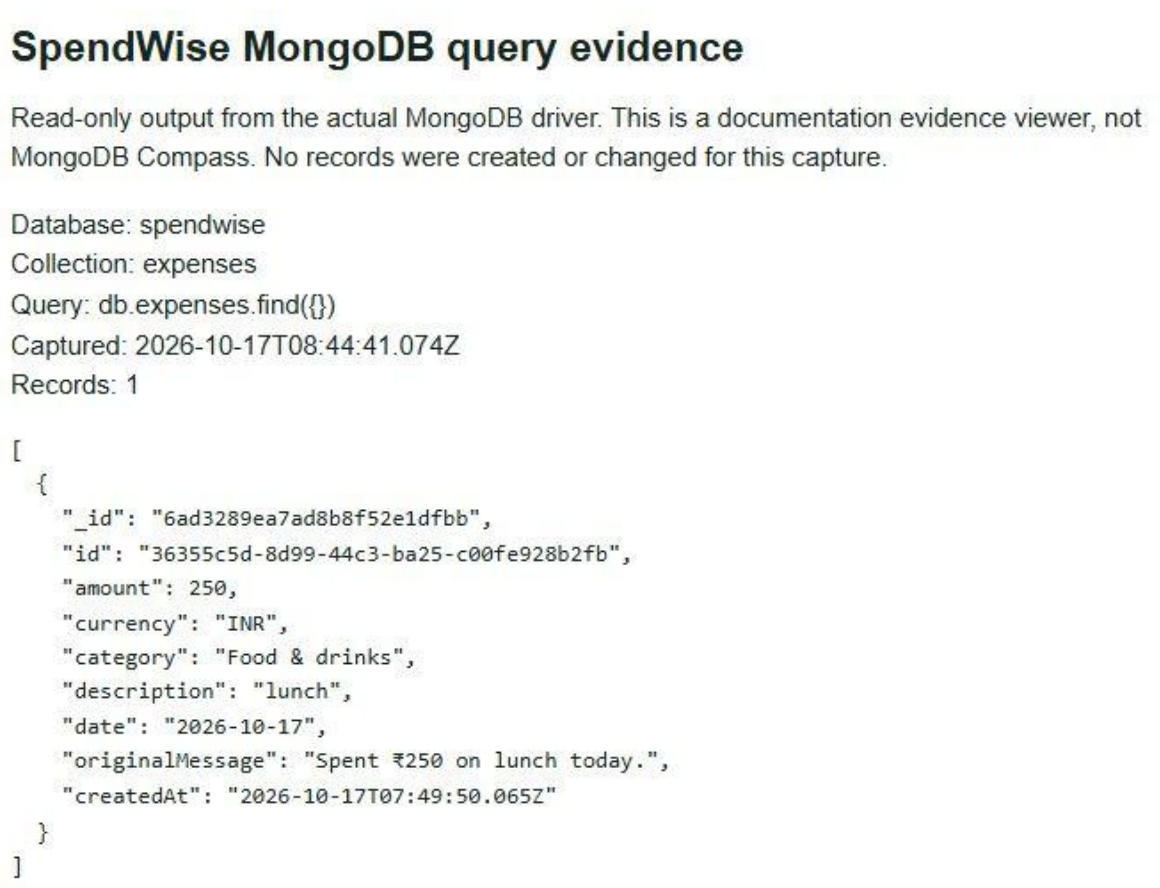


Figure 6. Actual Settings page: n8n, Ollama and MongoDB all report Connected. This demonstrates local service availability; it is not a public deployment.

Actual MongoDB records



SpendWise MongoDB query evidence

Read-only output from the actual MongoDB driver. This is a documentation evidence viewer, not MongoDB Compass. No records were created or changed for this capture.

Database: spendwise
Collection: expenses
Query: db.expenses.find({})
Captured: 2026-10-17T08:44:41.074Z
Records: 1


```
[
  {
    "_id": "6ad3289ea7ad8b8f52e1dfbb",
    "id": "36355c5d-8d99-44c3-ba25-c00fe928b2fb",
    "amount": 250,
    "currency": "INR",
    "category": "Food & drinks",
    "description": "lunch",
    "date": "2026-10-17",
    "originalMessage": "Spent ₹250 on lunch today.",
    "createdAt": "2026-10-17T07:49:50.065Z"
  }
]
```


Figure 7. Read-only MongoDB driver query of spendwise.expenses. The documentation viewer explicitly identifies itself and displays the actual stored document, including _id, amount, currency, category, description, date, originalMessage and createdAt. It is not a fabricated MongoDB Compass interface.

The stored ID is 36355c5d-8d99-44c3-ba25-c00fe928b2fb. Its amount is 250, currency INR, category Food & drinks, description lunch and date 2026-10-17. The original message is “Spent ₹250 on lunch today.” The captured JSON also records the unique request-ID index and zero remaining integration-test records.

Evidence traceability

evidence/verification/local-health-and-history.json and mongodb-current-records.json contain the current read-only outputs. The two mongodb-*-results.json files preserve earlier isolated test results. evidence/code/manifest.json maps every report listing to its source path, line range and SHA-256 hash. evidence/manifest.json lists the supplied evidence file hashes. Original screenshots are retained without cosmetic alteration. No .env file, token, password or private backup is included.

8. Conclusion

SpendWise AI demonstrates a working constrained AI expense workflow. The existing local model and n8n automation were retained, MongoDB became the primary expense database, and the UI consumes the same persisted records for confirmations, history and charts. Native schema output is followed by deterministic checks; the database readback, rather than an AI assertion, determines save success.

Limitations and planned enhancements

The current application supports one INR expense at a time, a limited set of explicit date forms and eight categories. Category/description interpretation can still be wrong and must be reviewed. Unspecified dates default to today. History is retrieved as a collection and processed in the frontend, so large datasets would need server-side pagination and aggregation.

This is a local single-user prototype. Multi-user authentication, hosted deployment, bank feeds, recurring transactions, in-app expense editing/deletion, automatic budget notifications and open-ended financial advice are not implemented. Budgets and retry receipts use local backend files. No historical spreadsheet import, backup restore drill, load test or comprehensive mobile-device test is claimed.

Possible future work includes explicit user review before saving, safe edit/delete flows, broader date/currency support, authenticated multi-user access, server-side analytics, database backup/restore testing and a verified hosted deployment. These are planned enhancements, not completed features.

Submission items requiring student input

Ojas Verma's supplied academic details are included. The student must complete the course duration, actual course progress/completion details and certificate links or attachments in Section 3. The project repository URL is verified and linked in Section 7. A public deployment URL is absent; the report states local-only operation. Confirm any institution-specific cover-page, declaration, signature or certificate-upload requirement separately; none is invented.

Official technology references

The official links below were opened successfully on 17 October 2026. They explain the technology capabilities, while the project's source and evidence establish what this application actually implements.

Ollama structured outputs

n8n MongoDB node

MongoDB schema validation

React documentation

Express documentation

Docker Compose documentation

Simple project explanation

You describe an expense. The local model extracts the details. Code checks the amount and date. n8n stores the expense in MongoDB and reads it back. Only then does the application say saved. The dashboard reads those same records. The final documentation preserves this evidence and clearly marks the remaining certificate and deployment items.